SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

ASSIGNMENT 1

PYTHON PROGRAMMING

Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative

Done By: Deepika Gopu

Register Number: 1911260133

Course Outcome: CO1 — Python Programming

Course code-CSA0803

GITHUB LINK- https://github.com/gtdeepika4-create/Python-.git

Problem Scenario

Ksheera Village Milk Producers' Co-operative collects milk twice daily from approximately 300 farmers. Each entry records farmer ID, litres, fat percentage, SNF percentage and shift. The base rate is ₹30.00 per litre at 3.5% fat and 8.5% SNF. Each 0.1% fat step changes the rate by ₹0.40 per litre, and each 0.1% SNF step changes it by ₹0.30 per litre. Payments are settled every ten days. The design should be accurate, validate input, reduce repeated rate values and scale towards 6,000 and 150,000 farmers.

General Design Assumptions

- Reference values are 3.5% fat and 8.5% SNF; fat is permitted from 2.0% through 7.0%.
- Milk quantity must be non-negative, SNF must be positive, farmer ID must not be blank, and shift must be Morning or Evening.
- Step counts are rounded to the nearest 0.1 percentage point. For actual cooperative deployment, the official rounding rule must be confirmed.
- Money should be calculated with Decimal and rounded to two decimal places using an explicitly chosen rounding rule.
- The report includes screenshot placeholders; replace them with real screenshots from your own Python execution. Do not treat placeholders as execution evidence.

1) TASK- 1-Analysis of the Milk Farmer Payment Problem

1. Inputs and Outputs

Inputs: Farmer ID (string), litres (float/Decimal), fat percentage (float/Decimal), SNF percentage (float/Decimal), shift (string), and ten daily entries for the payment period.

Outputs: rate per litre, payment amount for each valid entry, and the ten-day total for the same farmer.

2. Constants

- BASE_RATE = 30.00 — base price per litre.
- FAT_STEP_VALUE = 0.40 — adjustment per 0.1% fat step.
- SNF_STEP_VALUE = 0.30 — adjustment per 0.1% SNF step.
- REFERENCE_FAT = 3.5; REFERENCE_SNF = 8.5.
- MIN_FAT = 2.0; MAX_FAT = 7.0; DAYS = 10; MIN_LITRES = 0.

3. Data Types

Value	Type	Reason
Farmer ID / shift	String	Text identifiers
Number of farmers / days	Integer	Whole-number counts
Litres, fat, SNF, rate	Decimal or float	May contain fractional values
Payment total	Decimal	Exact decimal monetary arithmetic
Validation result	Boolean	True/False condition

4. Validation Rules

- Farmer ID must not be empty; shift must be Morning or Evening.
- Litres must be greater than or equal to zero.
- Fat must be between 2.0% and 7.0%, inclusive.
- SNF must be positive and numeric.
- Only validated entries are included in the ten-day total.

5. Assumptions

- Each farmer has a unique ID and may submit milk in both shifts.
- A ten-day total includes valid payments for one farmer only.
- The rate formula supplied in the assignment is applied consistently.
- Money is rounded according to a confirmed cooperative policy.

6. Alternative Design and Complexity Analysis

Using separate variables (litres1, fat1, snf1, etc.) becomes repetitive. For 300 farmers, three values alone require 900 separately named variables, not counting IDs and shifts. A list of tuples stores each record together and can be processed in a loop. For n records, both approaches require O(n) storage if every record is retained, and processing all records takes O(n) time. The list-based approach is more scalable and maintainable; a running total can reduce extra working storage to O(1) if records do not need to be retained.

7. Conclusion

Named constants, clear validation and list-based records make the payment calculator easier to update and scale. Decimal is preferred for monetary totals.

2) TASK-2-Explore Values and Types in the Python Interpreter

1. Data Types and Arithmetic

Expression	Result / type
type(10)	int
type(10.0)	float
type("10")	str
type(True)	bool
type([10, 8.5])	list
7 / 2	3.5
7 // 2	3
7 % 2	1
-7 // 2	-4

The / operator performs true division, // performs floor division, and % returns the remainder. Floor division rounds down toward negative infinity, so -7 // 2 is -4.

2. Floating-Point Precision

The expression 0.1 + 0.2 == 0.3 evaluates to False because binary floating-point cannot represent many decimal fractions exactly. Small errors can accumulate in financial totals.

3. Completed TODO 1 — Decimal Total

```
from decimal import Decimal
import sys

values = [10, 10.0, "10", True, [10, 8.5]]
for value in values:
    print(repr(value), "->", type(value).__name__)

print("7 / 2 =", 7 / 2)
print("7 // 2 =", 7 // 2)
print("7 % 2 =", 7 % 2)
print("-7 // 2 =", -7 // 2)
print("0.1 + 0.2 == 0.3:", 0.1 + 0.2 == 0.3)

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
print("sizes:", sys.getsizeof(0), sys.getsizeof(0.0),
      sys.getsizeof(Decimal("33.40")))

# Integer paise is exact when each amount has at most two decimal places.
total_paise = sum(int(Decimal(amount) * 100) for amount in payments)
print("Integer paise:", total_paise, "paise =", f"Rs {total_paise / 100:.2f}")
```

Expected key output: float total is approximately 33400.30000000061; Decimal total is 33400.30; the comparison is False for float and True for Decimal. Creating Decimal from a string preserves the

decimal value. Decimal(float(amount)) imports the float approximation and therefore cannot recover the original exact decimal.

4. Alternative Methods and Complexity

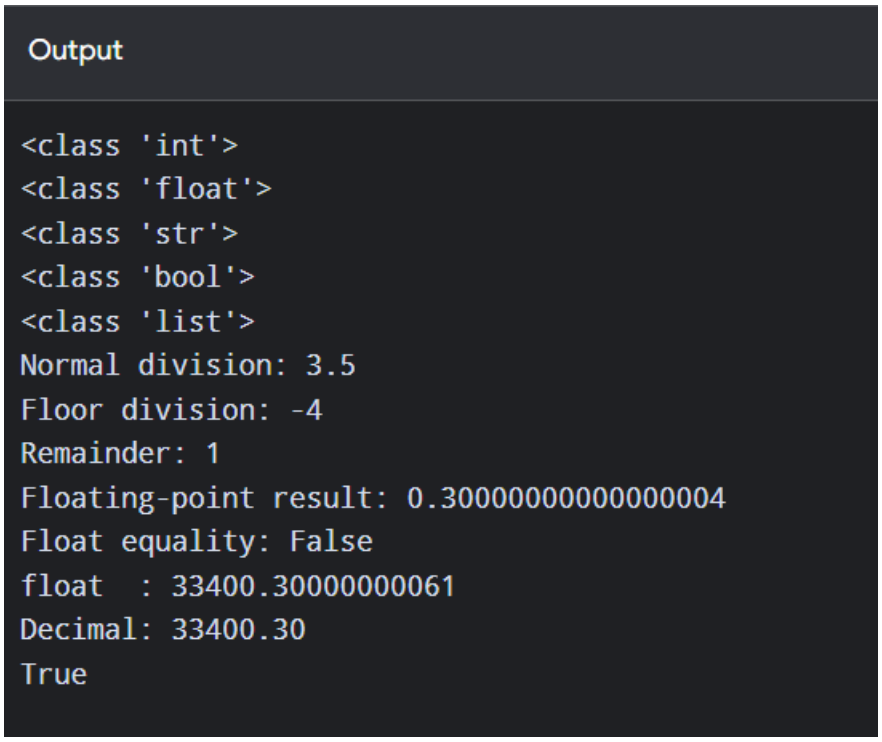
Method	Time per addition	Exact for cents?	Notes
Decimal from strings	O(1)	Yes, when used correctly	Clear monetary semantics; more memory/time than float.
Integer paise	O(1)	Yes, for values representable in paise	Convert ₹33.40 to 3340 paise; convert back at display.
float + round each step	O(1)	Not guaranteed	Rounding reduces visible error but binary representation remains approximate.

Total time is O(n). Running-total extra space is O(1), excluding the input list. sys.getsizeof() reports implementation-dependent sizes; run the program on the installed Python version and record the displayed sizes.

5. Conclusion

Decimal created from strings or integer paise is safer for money than binary float. The code above is the executable evidence for the type, arithmetic and precision experiments.

OUTPUT SCREENSHOT



3) TASK-3-Choose Variables and Name the Constants

1. Naming Variables and Constants

Python identifiers begin with a letter or underscore and cannot contain spaces or be reserved keywords. Constants are conventionally written in uppercase.

```
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
REFERENCE_FAT = 3.5
REFERENCE_SNF = 8.5
MIN_FAT = 2.0
MAX_FAT = 7.0

RATE_CARD = {"base": BASE_RATE, "fat": FAT_STEP_VALUE, "snf": SNF_STEP_VALUE}
print("Rate card:", RATE_CARD)

value = 10
print(value, type(value).__name__)
value = "ten"
print(value, type(value).__name__)

a, b = 6, 9
print("Before swap:", a, b)
a, b = b, a
print("After swap:", a, b)
```

2. Why Python Has No True Constants

Python does not prevent reassignment; uppercase names communicate that a value is intended to remain unchanged. Dynamic typing allows a variable to refer first to an integer and later to a string. Multiple assignment can swap values using a, b = b, a.

3. Alternative Design and Complexity

- Magic numbers must be found and updated at every occurrence; if there are k occurrences, as many as k edits may be needed.
- Named constants keep each rate in one place and improve readability.
- A dictionary such as RATE_CARD = {'base': 30.0, 'fat': 0.40, 'snf': 0.30} groups related values under descriptive keys.
- For a fixed number of rates, time and space are effectively O(1) for each design; dictionary lookup is average O(1).

4. Conclusion

Named constants or a rate dictionary make rate revisions safer and reduce repeated code.

OUTPUT SCREENSHOT

Output

```
Base rate: 30.0
Fat adjustment: 0.4
SNF adjustment: 0.3
10 <class 'int'>
Milk <class 'str'>
After swapping: 20 10
{'base': 30.0, 'fat': 0.4, 'snf': 0.3}
```

4) TASK-4-Write Correct Expressions

1. Expressions

fat_steps = round((fat - 3.5) / 0.1); snf_steps = round((snf - 8.5) / 0.1); rate = 30.00 + fat_steps × 0.40 + snf_steps × 0.30; amount = litres × rate.

2. Operator Precedence and Unexpected Results

- Parentheses are evaluated first; exponentiation precedes unary minus, so -2**2 is -4 while (-2)**2 is 4.
- (4.2 - 3.5) / 0.1 can be 7.000000000000001 due to floating-point representation; round() yields the intended step count 7.
- int(-2.7) is -2 because int truncates toward zero; math.floor(-2.7) is -3 because floor rounds down.
- 2.0 <= fat <= 7.0 checks both boundaries inclusively. and requires both conditions, or requires at least one, and not negates a condition.

```
import math

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
REFERENCE_FAT = 3.5
REFERENCE_SNF = 8.5

def rate_per_litre(fat, snf, step_method=round):
    fat_steps = step_method((fat - REFERENCE_FAT) / 0.1)
    snf_steps = round((snf - REFERENCE_SNF) / 0.1)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return fat_steps, snf_steps, round(rate, 2)
```

```
for fat in (4.2, 3.1):
    print("fat =", fat)
    for name, method in (("round", round), ("int", int), ("floor", math.floor)):
        fs, ss, rate = rate_per_litre(fat, 8.5, method)
        print(name, "fat_steps =", fs, "rate =", f"{rate:.2f}")

print("(4.2 - 3.5) / 0.1 =", (4.2 - 3.5) / 0.1)
print("round((4.2 - 3.5) / 0.1) =", round((4.2 - 3.5) / 0.1))
print("-2**2 =", -2**2, "(-2)**2 =", (-2)**2)
print("int(-2.7) =", int(-2.7), "floor(-2.7) =", math.floor(-2.7))
fat = 4.2
print("fat in range:", 2.0 <= fat <= 7.0)
```

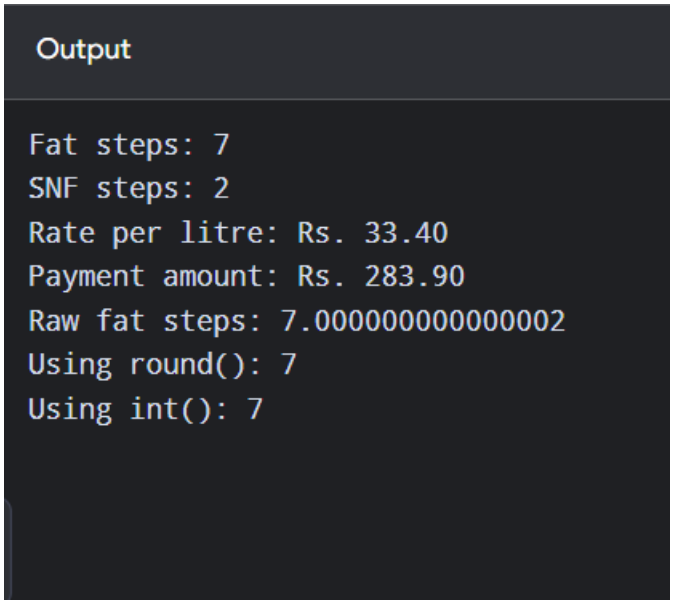
3. Alternative Methods

For fat = 4.2%, the step calculation is approximately 7; round(), int() and floor() all produce 7. For fat = 3.1%, the calculation is approximately -4; round() and floor() produce -4 while int() truncates to -3. The latter changes the rate adjustment by one step. Each method uses O(1) time and O(1) extra space.

4. Conclusion

Use an explicitly selected step-rounding rule and validate values before calculating. The cooperative should confirm how exact half-steps are treated.

OUTPUT SCREENSHOT



5) TASK -5-Read Input and Convert Types

input() always returns text. int() converts whole-number strings and float() converts standard decimal-number strings. A conversion such as float('8,5') or float('eight') raises ValueError; surrounding spaces such as float(' 8.5 ') are accepted.

```
def read_float(prompt):
    while True:
        raw = input(prompt)
        try:
            return float(raw)
        except ValueError:
            print("Invalid number. Enter a value such as 8.5 (not '8,5' or 'eight').")

try:
    litres = read_float("Enter litres: ")
    fat = read_float("Enter fat percentage: ")
    snf = read_float("Enter SNF percentage: ")

    if litres < 0:
        print("Invalid entry: litres cannot be negative.")
    elif not 2.0 <= fat <= 7.0:
        print("Invalid entry: fat must be between 2.0 and 7.0.")
    elif snf <= 0:
        print("Invalid entry: SNF must be positive.")
    else:
        print(f"Litres: {litres:8.2f}")
        print(f"Fat:    {fat:8.2f}")
        print(f"SNF:    {snf:8.2f}")
except KeyboardInterrupt:
    print("\nInput cancelled.")
```

Alternative Design and Complexity

String checks can reject some malformed inputs but may miss valid numeric formats or require many special cases. try/except ValueError attempts conversion directly and handles failures safely. For an input string of length m, parsing is O(m), with O(1) additional storage for a numeric value, excluding the input string. The f-string format {amount:8.2f} reserves eight character positions and displays two decimal places.

Conclusion

Convert input explicitly, handle conversion errors, validate ranges and display values consistently.

OUTPUT SCREENSHOT —

VALID INPUT

```
Output

Enter Farmer ID: F001
Enter milk quantity in litres: 10
Enter fat percentage: 3.5
Enter SNF percentage: 8.5

--- Milk Payment Details ---
Farmer ID: F001
Milk Quantity: 10.0 litres
Fat Percentage: 3.5 %
SNF Percentage: 8.5 %
Rate per litre: Rs. 30.00
Total Payment: Rs. 300.00
```

INVALID INPUT

```
Output

Enter Farmer ID: F004
Enter milk quantity in litres: -4
Enter fat percentage: 3.5
Enter SNF percentage: 8.5
Invalid input: Milk quantity must be greater than zero.
```

6)TASK-6- Organise the Logic into Functions

BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
REFERENCE_FAT = 3.5
REFERENCE_SNF = 8.5

def rate_per_litre(fat, snf):

```
    """Calculate the rate per litre from fat and SNF percentages."""
    fat_steps = round((fat - REFERENCE_FAT) / 0.1)
    snf_steps = round((snf - REFERENCE_SNF) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE
                 + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return the payment for one milk entry."""
    return round(litres * rate_per_litre(fat, snf), 2)

def ten_day_total(payments):
    """Return the sum of payments for the ten-day period."""
    return round(sum(payments), 2)

payments = [
    payout(10, 4.0, 8.5),
    payout(5, 3.5, 8.5),
    payout(8, 3.1, 8.3),
]
print("Payments:", payments)
print("Ten-day total:", f"{ten_day_total(payments):.2f}")
```

Explanation

- rate_per_litre() calculates and returns the rate; payout() returns one payment; ten_day_total() returns the sum of payments.
- return sends a value back to the caller; print() only displays it.
- Parameters and variables created inside a function are local. Constants defined at module level are global names and can be read inside functions.
- Docstrings describe each function and should appear immediately after the function definition.

Alternative Design and Complexity

A single long script may repeat logic and be harder to debug. Functions support reuse and testing. Lambda expressions are suitable for short expressions but are less suitable for multi-step validation and documentation. For n entries, processing is O(n) time; storing all payments uses O(n) space, while a running total uses O(1) extra space. Function-call overhead is constant per call.

Conclusion

Small named functions improve readability, reuse and testability.

OUTPUT SCREENSHOT —

Output

```
Payment: Rs. 300.00
Payment: Rs. 283.90
Payment: Rs. 333.60
Ten-day total: Rs. 917.50
```

7) TASK-7-Build and Use a Python Module

Save the following module as milkrate.py. The second file, Task07_import_example.py, imports payout(). Keep both files in the same folder.

```
"""Rate card and payment functions for the village milk co-operative."""
BASE_RATE = 30.00
FAT_STEP_VALUE = 0.40
SNF_STEP_VALUE = 0.30
REFERENCE_FAT = 3.5
REFERENCE_SNF = 8.5

def rate_per_litre(fat, snf):
    """Return the price per litre rounded to two decimal places."""
    fat_steps = round((fat - REFERENCE_FAT) / 0.1)
    snf_steps = round((snf - REFERENCE_SNF) / 0.1)
    return round(BASE_RATE + fat_steps * FAT_STEP_VALUE
                 + snf_steps * SNF_STEP_VALUE, 2)

def payout(litres, fat, snf):
    """Return the payment amount rounded to two decimal places."""
    return round(litres * rate_per_litre(fat, snf), 2)

if __name__ == "__main__":
    for litres, fat, snf in [
        (10, 3.5, 8.5),
        (8.5, 4.2, 8.7),
        (12, 3.1, 8.3),
    ]:
        rate = rate_per_litre(fat, snf)
        pay = payout(litres, fat, snf)
```

```
        print(f"{litres:5} L  fat {fat}  snf {snf}"
              f"  rate {rate:6.2f}  pay {pay:8.2f}")
```

Second script:

```
from milkrate import payout


print("Imported payment:", f"{payout(10, 3.5, 8.5):.2f}")
```

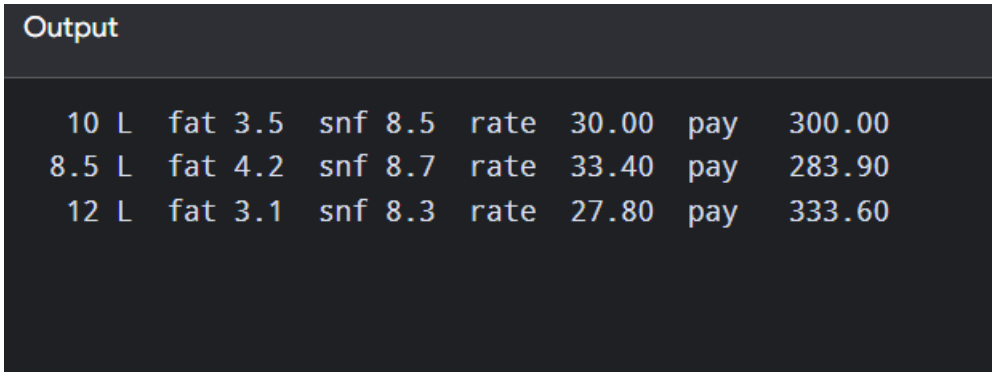
Explanation

- import milkrate uses names through milkrate.payout(); from milkrate import payout imports that name directly; import milkrate as mr uses the alias mr.
- if __name__ == '__main__' runs the demonstration only when the file is executed directly. When imported, __name__ is the module name, so the demonstration does not run.
- math supplies mathematical functions; decimal supports decimal arithmetic. Python searches the current module directory and paths listed in sys.path.
- Python caches loaded modules in sys.modules, so later imports in the same process normally reuse the loaded module. Importing has a one-time loading cost; each payment calculation is O(1).

Expected Output

```
 10 L  fat 3.5  snf 8.5  rate  30.00  pay   300.00
8.5 L  fat 4.2  snf 8.7  rate  33.40  pay   283.90
 12 L  fat 3.1  snf 8.3  rate  27.80  pay   333.60
```

OUTPUT SCREENSHOT



8) TASK-8-Plan the Algorithm Before Coding

1. Problem-Solving Stages

- Understand: identify inputs, formula, outputs and invalid cases.
- Plan: write the steps, validations and total calculation.
- Write: translate pseudocode into Python functions.
- Test: compare results against hand calculations and invalid cases.

2. Pseudocode

```
BEGIN
  total ← 0
```

```
    FOR day ← 1 TO 10
      INPUT farmer_id, litres, fat, snf, shift
      IF farmer_id is not empty AND litres >= 0
        AND 2.0 <= fat <= 7.0 AND snf > 0
        AND shift is Morning or Evening THEN
         fat_steps ← ROUND((fat - 3.5) / 0.1)
         snf_steps ← ROUND((snf - 8.5) / 0.1)
         rate ← 30.00 + 0.40 × fat_steps + 0.30 × snf_steps
         payment ← ROUND(litres × rate, 2)
         DISPLAY payment
         total ← total + payment
      ELSE
         DISPLAY "Invalid Entry"
      END IF
    END FOR
    DISPLAY total
  END
```

3. Flowchart

Start → Read entry → Validate ID, litres, fat, SNF and shift → [Invalid?] Yes: display error and continue; No: calculate steps → calculate rate → calculate payment → add to ten-day total → [More entries?] Yes: read next entry; No: display total → End.

4. Hand Trace for Three Entries

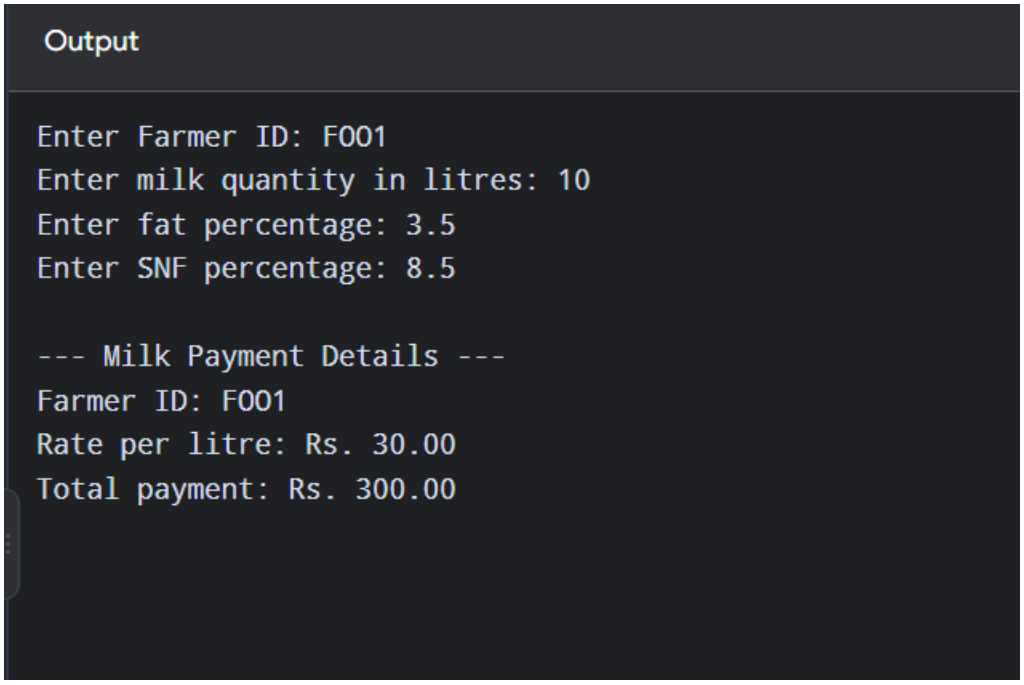
Entry	Litres	Fat	SNF	Calculation / status	Payment
1	10	4.0	8.0	Rate = 30 + 2(0.40) - 5(0.30) = 29.30	293.00
2	5	3.0	7.0	Rate = 30 - 5(0.40) - 15(0.30) = 23.50	117.50
3	8	8.0	7.0	Fat is outside 2.0–7.0; reject	Invalid

Valid total = ₹410.50.

5. Alternative Design and Complexity

Storing all n entries and then summing takes O(n) time and O(n) storage. A running total also takes O(n) time but only O(1) extra storage. Keep the records when they must be audited, corrected or reprocessed.

OUTPUT SCREENSHOT



9)TASK -9- Implement and Test the Calculator

The following program uses Decimal, validation, explicit rounding and automated tests. Run it and use its actual output as evidence.

```
from decimal import Decimal, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
REFERENCE_FAT = Decimal("3.5")
REFERENCE_SNF = Decimal("8.5")
CENT = Decimal("0.01")

def money(value):
    return value.quantize(CENT, rounding=ROUND_HALF_UP)

def payment(litres, fat, snf):
    """Calculate a payment; input values are converted from strings to Decimal."""
    litres, fat, snf = Decimal(str(litres)), Decimal(str(fat)), Decimal(str(snf))
    if litres < 0:
        raise ValueError("Litres cannot be negative")
    if not Decimal("2.0") <= fat <= Decimal("7.0"):
        raise ValueError("Fat must be between 2.0 and 7.0")
    if snf <= 0:
        raise ValueError("SNF must be positive")
    fat_steps = ((fat - REFERENCE_FAT) / Decimal("0.1")).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
```

```
    snf_steps = ((snf - REFERENCE_SNF) / Decimal("0.1")).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)
    rate = BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE
    return money(litres * rate)

tests = [
    ("above standard", 10, 4.0, 9.0, Decimal("346.00")),
    ("below standard", 5, 3.0, 7.5, Decimal("165.00")),
    ("standard boundary", 10, 3.5, 8.5, Decimal("300.00")),
    ("minimum fat", 1, 2.0, 8.5, Decimal("24.00")),
    ("maximum fat", 1, 7.0, 8.5, Decimal("44.00")),
]
for label, litres, fat, snf, expected in tests:
    actual = payment(litres, fat, snf)
    print(label, "expected =", expected, "actual =", actual,
          "PASS" if actual == expected else "FAIL")

for label, litres, fat, snf in [
    ("negative litres", -1, 3.5, 8.5),
    ("fat too low", 1, 1.9, 8.5),
    ("fat too high", 1, 7.1, 8.5),
]:
    try:
        payment(litres, fat, snf)
    except ValueError as exc:
        print(label, "PASS:", exc)
    else:
        print(label, "FAIL")

five_entries = [(10, 4, 8), (5, 3, 7), (8, 5, 9),
                (6, 2, 8.5), (4, 7, 8.5)]
total = sum((payment(*entry) for entry in five_entries), Decimal("0.00"))
print("Five-entry total:", money(total))
assert payment(10, 3.5, 8.5) == Decimal("300.00")
print("Automated assertions passed.")
```

Test Plan

Test case	Input	Expected behaviour
Above standard	10 L, fat 4.0, SNF 9.0	Valid payment ₹346.00
Below standard	5 L, fat 3.0, SNF 7.5	Valid payment ₹165.00
Reference values	10 L, fat 3.5, SNF 8.5	₹300.00
Minimum fat	1 L, fat 2.0, SNF 8.5	₹24.00
Maximum fat	1 L, fat 7.0, SNF 8.5	₹44.00

Negative litres	-1 L	Reject with ValueError
Fat too low / high	1.9% / 7.1%	Reject with ValueError
Invalid text	'eight' for a number	Input conversion should reject it

Manual vs Automated Testing

Manual testing requires entering values and checking results individually. Automated tests use a test function and assert statements to rerun the same cases quickly after code changes. Both take O(n) time for n test cases; automated tests reduce repeated effort.

OUTPUT SCREENSHOT —

FOR VALID INPUT

Output

```
Enter the number of milk entries: 2

Farmer 1
Enter Farmer ID: F001
Enter shift (Morning/Evening): MORNING
Enter milk quantity in litres: 10
Enter fat percentage: 3.5
Enter SNF percentage: 8.5
Farmer ID: F001
Rate per litre: Rs. 30.00
Payment: Rs. 300.00

Farmer 2
Enter Farmer ID: F002
Enter shift (Morning/Evening): Evening
Enter milk quantity in litres: 8.5
Enter fat percentage: 4.2
Enter SNF percentage: 8.7
Farmer ID: F002
Rate per litre: Rs. 33.40
Payment: Rs. 283.90
```

FOR INVALID INPUT

```
Output

Enter the number of milk entries: 1

Farmer 1
Enter Farmer ID: F002
Enter shift (Morning/Evening): Morning
Enter milk quantity in litres: -9
Enter fat percentage: 3.5
Enter SNF percentage: 8.5
Invalid milk quantity. Entry rejected.

Total payment for valid entries: Rs. 0.00
```

10)TASK-10- Compare Expectations with Execution

Run the benchmark below on your own computer. Execution times depend on hardware, Python version and system load, so record the values printed on your machine rather than copying invented timings.

```
from decimal import Decimal
from time import perf_counter

def total_float(values):
    total = 0.0
    for value in values:
        total += float(value)
    return total

def total_decimal(values):
    total = Decimal("0.00")
    for value in values:
        total += Decimal(value)
    return total

for count in (300, 6000, 150000):
    values = ["33.40"] * count

    start = perf_counter()
    float_total = total_float(values)
    float_seconds = perf_counter() - start
```

```
    start = perf_counter()
    decimal_total = total_decimal(values)
    decimal_seconds = perf_counter() - start

    print(f"n={count}")
    print(f"float total={float_total:.2f}, time={float_seconds:.6f}s")
    print(f"Decimal total={decimal_total:.2f}, time={decimal_seconds:.6f}s")
    print()

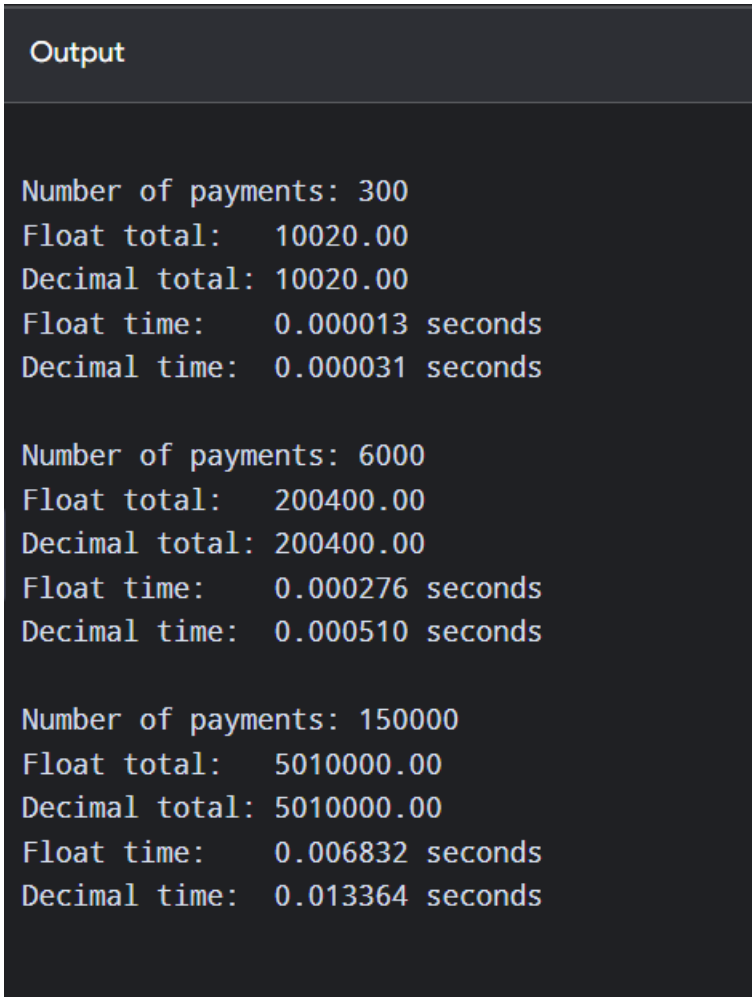
# Compare rounding every payment with rounding only the final sum.
amounts = [Decimal("0.005")] * 10
round_each = sum((x.quantize(Decimal("0.01")) for x in amounts), Decimal("0.00"))
round_final = sum(amounts, Decimal("0.00")).quantize(Decimal("0.01"))
print("Round each:", round_each, "Round final:", round_final)
```

How to Interpret Results

- Compare totals for 300, 6,000 and 150,000 entries. Float may show small representation error; Decimal is exact for decimal inputs used here.
- time.perf_counter() measures elapsed wall-clock time. Repeat the benchmark several times and compare typical results.
- Rounding each payment and rounding only the final total can differ when values have fractions of a cent. The cooperative must specify its settlement rule.
- A rate dictionary can cache rates by (fat, snf) key. Average lookup is O(1); storing k distinct pairs uses O(k) extra space. Recalculation is O(1) per entry too, so caching helps mainly when rate computation is repeated and measured to be costly.

Number of payments	Float total (expected)	Decimal total (expected)	Measured times
300	10020.00	10020.00	Fill from program output
6,000	200400.00	200400.00	Fill from program output
150,000	5010000.00	5010000.00	Fill from program output

OUTPUT SCREENSHOT —



11) TASK -11-Recommend and Defend the Final Design

1. Recommended Design

- Use Decimal values created from input strings for monetary calculations, or integer paise when all values are limited to two decimal places.
- Round according to an explicit cooperative-approved policy, rather than relying on default float behaviour.
- Keep rates as named constants or load a validated JSON rate card at startup.
- Validate all input before calculating and maintain an auditable record of entries and payment totals.
- For 150,000 farmers, use a database, access controls, transaction logging, backups, monitoring and performance tests rather than relying only on an in-memory list.

2. Limitations

Decimal can be slower than float; incorrect rate files can produce incorrect totals; missing validation can admit bad data; and a small demonstration program does not provide database transactions, security, concurrency control or backups.

3. Alternative — JSON Rate File

```
import json
from pathlib import Path

# A real submission can keep this content in a separate rates.json file.
rates = {
    "base_rate": 30.00,
    "fat_step_value": 0.40,
    "snf_step_value": 0.30,
    "reference_fat": 3.5,
    "reference_snf": 8.5
}
Path("rates.json").write_text(json.dumps(rates, indent=2), encoding="utf-8")

with open("rates.json", encoding="utf-8") as file:
    loaded_rates = json.load(file)

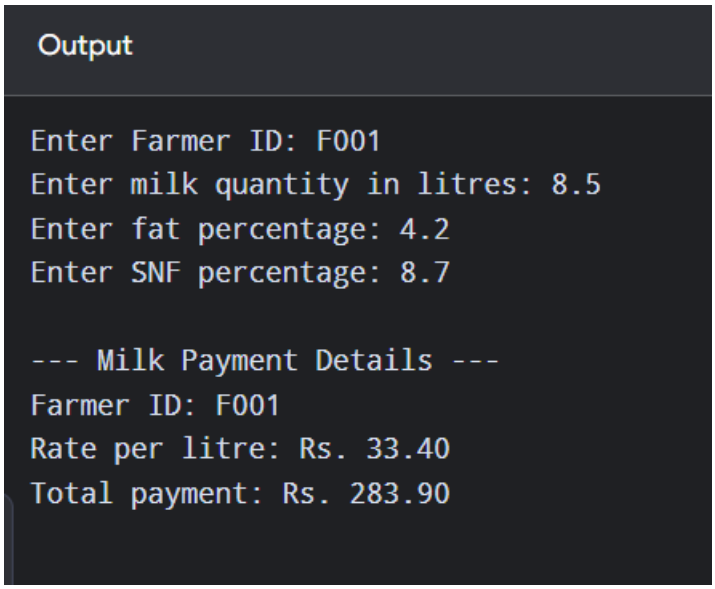
print("Loaded rate card:")
for key, value in loaded_rates.items():
    print(f"{key}: {value}")
```

Loading t rate values takes approximately O(t) time and O(t) memory. A separate rate file makes revisions easier because the calculation code need not be edited for every rate change. The file must be validated and protected from accidental changes.

4. Conclusion

The recommended solution combines Decimal or integer paise, named rate data, explicit validation, reusable functions, automated tests and database-backed storage at district scale. Support for 150,000 farmers should be confirmed through load testing.

OUTPUT SCREENSHOT —



12) TASK-12-Day-4 Design Challenge: Re-evaluate and Adapt the Design

A. Impact Analysis — 2 Marks

The analyser supplies fat and SNF to two decimal places. From April through June, the base rate changes to ₹32.00 and each fat step changes to ₹0.45. A bonus of ₹0.50 per litre is paid only when a farmer's ten-day total exceeds 500 litres. The rate card, rate function and ten-day total function must therefore accept a season and compute a bonus. Step rounding for hundredth-percentage readings must be explicitly defined.

B. Evaluation of Original Design — 2 Marks

The original design is modular enough to retain, but fixed constants are insufficient. A worked seasonal example with 10 L, fat 3.75% and SNF 8.70% gives fat difference 0.25 percentage points and SNF difference 0.20. Under nearest-step rounding with ROUND_HALF_UP, the fat step count is 3 and SNF step count is 2; the seasonal rate is ₹32.00 + 3×₹0.45 + 2×₹0.30 = ₹33.95, so the payment is ₹339.50. If the cooperative instead defines only completed steps, the fat adjustment would be different; the rule must be confirmed.

C. Adaptation / Revised Code — 2 Marks

```
from decimal import Decimal, ROUND_HALF_UP

CENT = Decimal("0.01")
REFERENCE_FAT = Decimal("3.50")
REFERENCE_SNF = Decimal("8.50")
SNF_STEP_VALUE = Decimal("0.30")
BONUS_PER_LITRE = Decimal("0.50")

def rounded_steps(value, reference):
    """Round step count to nearest 0.1 percentage point."""
    return ((value - reference) / Decimal("0.1")).quantize(
        Decimal("1"), rounding=ROUND_HALF_UP)

def rate_per_litre(fat, snf, seasonal=False):
    """Use seasonal rates from April through June when seasonal=True."""
    base = Decimal("32.00") if seasonal else Decimal("30.00")
    fat_step_value = Decimal("0.45") if seasonal else Decimal("0.40")
    fat_steps = rounded_steps(Decimal(str(fat)), REFERENCE_FAT)
    snf_steps = rounded_steps(Decimal(str(snf)), REFERENCE_SNF)
    return base + fat_steps * fat_step_value + snf_steps * SNF_STEP_VALUE

def ten_day_payment(entries, seasonal=False):
    """Return total payment and bonus for a farmer's ten-day entries."""
    total_litres = sum(Decimal(str(entry[0])) for entry in entries)
    base_total = sum(
        Decimal(str(litres)) * rate_per_litre(fat, snf, seasonal)
        for litres, fat, snf in entries
```

```
    )
    bonus = total_litres * BONUS_PER_LITRE if total_litres > 500 else Decimal("0.00")
    return base_total.quantize(CENT, rounding=ROUND_HALF_UP), bonus.quantize(
        CENT, rounding=ROUND_HALF_UP)

if __name__ == "__main__":
    sample = [(10, "3.75", "8.70")]
    for seasonal in (False, True):
        base_total, bonus = ten_day_payment(sample, seasonal)
        print("Seasonal rates:", seasonal,
              "base payment:", base_total, "bonus:", bonus,
              "total:", base_total + bonus)

    print("501-litre bonus:", (Decimal("501") * BONUS_PER_LITRE).quantize(CENT))
    print("500-litre bonus:", Decimal("0.00"))
```

D. Evidence-Based Defence — 2 Marks

The revised design preserves reusable functions and adds seasonal rate selection plus a separate ten-day bonus. It prints results for original and seasonal rates and demonstrates the threshold: 501 litres earns ₹250.50, while exactly 500 litres earns no bonus. The sample program's nearest-step convention is explicit; verify that convention against the cooperative's official policy before deployment.

Complexity and Conclusion

Each entry is processed in O(1) time, so n entries take O(n) time. A running total uses O(1) extra space; retaining all entries uses O(n). Modifying the original design is preferable to rewriting it, because its functions and validation can be reused while rate selection and bonus rules are added.

OUTPUT SCREENSHOT — Insert the output of Task12_seasonal_design.py, including 500 L and 501 L bonus tests.

